

Practical_3_24CS099

Environment Setup & Local Data Loading

```
import os
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report
)
from imblearn.over_sampling import SMOTE

warnings.filterwarnings('ignore')
plt.style.use("ggplot")

# Load local dataset directly
df = pd.read_csv("diabetes.csv")

print("First 5 records:")
print(df.head())
```

First 5 records:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1

3	0.167	21	0
4	2.288	33	1

Dataset Dimensions

```
print("rows:", df.shape[0])
print("columns:", df.shape[1])
print("Shape:", df.shape)
```

```
rows: 768
columns: 9
Shape: (768, 9)
```

Summary Statistics

```
df.describe(include="all")

{"summary": "{\n  \"name\": \"df\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"Pregnancies\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 269.85223453356366,\n        \"min\": 0.0,\n        \"max\": 768.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          3.8450520833333335,\n          3.0,\n          768.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Glucose\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 243.73802348295857,\n        \"min\": 0.0,\n        \"max\": 768.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          120.89453125,\n          117.0,\n          768.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"BloodPressure\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 252.85250535810619,\n        \"min\": 0.0,\n        \"max\": 768.0,\n        \"num_unique_values\": 8,\n        \"samples\": [\n          69.10546875,\n          72.0,\n          768.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"SkinThickness\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 263.7684730531098,\n        \"min\": 0.0,\n        \"max\": 768.0,\n        \"num_unique_values\": 7,\n        \"samples\": [\n          768.0,\n          20.536458333333332,\n          32.0\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"Insulin\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 350.26059167945886,\n        \"min\": 0.0,\n        \"max\": 846.0,\n        \"num_unique_values\": 7,\n        \"samples\": [\n          768.0,\n          79.79947916666667,\n          127.25\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      },\n      \"column\": \"BMI\",\n      \"properties\": {\n        \"dtype\": \"number\",
```

```

{"std": 262.05117817552093, "min": 0.0, "max": 768.0, "num_unique_values": 8, "samples": [31.992578124999998, 32.0, 768.0], "semantic_type": "", "description": "", "column": "DiabetesPedigreeFunction", "properties": {"dtype": "number", "std": 271.3005221658502, "min": 0.078, "max": 768.0, "num_unique_values": 8, "samples": [0.47187630208333325, 0.3725, 768.0], "semantic_type": "", "description": "", "column": "Age", "properties": {"dtype": "number", "std": 260.1941178528413, "min": 11.76023154067868, "max": 768.0, "num_unique_values": 8, "samples": [33.240885416666664, 29.0, 768.0], "semantic_type": "", "description": "", "column": "Outcome", "properties": {"dtype": "number", "std": 271.3865920388932, "min": 0.0, "max": 768.0, "num_unique_values": 5, "samples": [0.3489583333333333, 1.0, 0.4769513772427971]}}, "semantic_type": "", "description": ""}
{"type": "dataframe"}

```

Data Types & Column Classification

```

print("Data Types:\n", df.dtypes)

categorical = df.select_dtypes(include="object").columns
numerical = df.select_dtypes(include=['int64', 'float64']).columns
print("\nCategorical columns:", list(categorical))
print("Numerical columns:", list(numerical))

```

```

Data Types:
Pregnancies      int64
Glucose           int64
BloodPressure    int64
SkinThickness    int64
Insulin           int64
BMI              float64
DiabetesPedigreeFunction  float64
Age              int64
Outcome          int64
dtype: object

```

```

Categorical columns: []
Numerical columns: ['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin', 'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome']

```

Missing Values & Uniqueness Inspection

```
print("Null values count:\n", df.isnull().sum())
print("\nEmpty string values count:\n", (df == " ").sum())
print("\nDuplicate records count:", df.duplicated().sum())
print("\nUnique values per column:\n", df.nunique())
```

Null values count:

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

Empty string values count:

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Outcome	0

dtype: int64

Duplicate records count: 0

Unique values per column:

Pregnancies	17
Glucose	136
BloodPressure	47
SkinThickness	51
Insulin	186
BMI	248
DiabetesPedigreeFunction	517
Age	52
Outcome	2

dtype: int64

Head & Tail Check

```
print("Head:")
print(df.head())
```

```
print("Tail:")
print(df.tail())
```

Head:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Tail:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI
763	10	101	76	48	180	32.9
764	2	122	70	27	0	36.8
765	5	121	72	23	112	26.2
766	1	126	60	0	0	30.1
767	1	93	70	31	0	30.4

	DiabetesPedigreeFunction	Age	Outcome
763	0.171	63	0
764	0.340	27	0
765	0.245	30	0
766	0.349	47	1
767	0.315	23	0

Dataset Information Overview

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
```

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	Pregnancies	768 non-null	int64
1	Glucose	768 non-null	int64
2	BloodPressure	768 non-null	int64
3	SkinThickness	768 non-null	int64
4	Insulin	768 non-null	int64
5	BMI	768 non-null	float64
6	DiabetesPedigreeFunction	768 non-null	float64
7	Age	768 non-null	int64
8	Outcome	768 non-null	int64

dtypes: float64(2), int64(7)

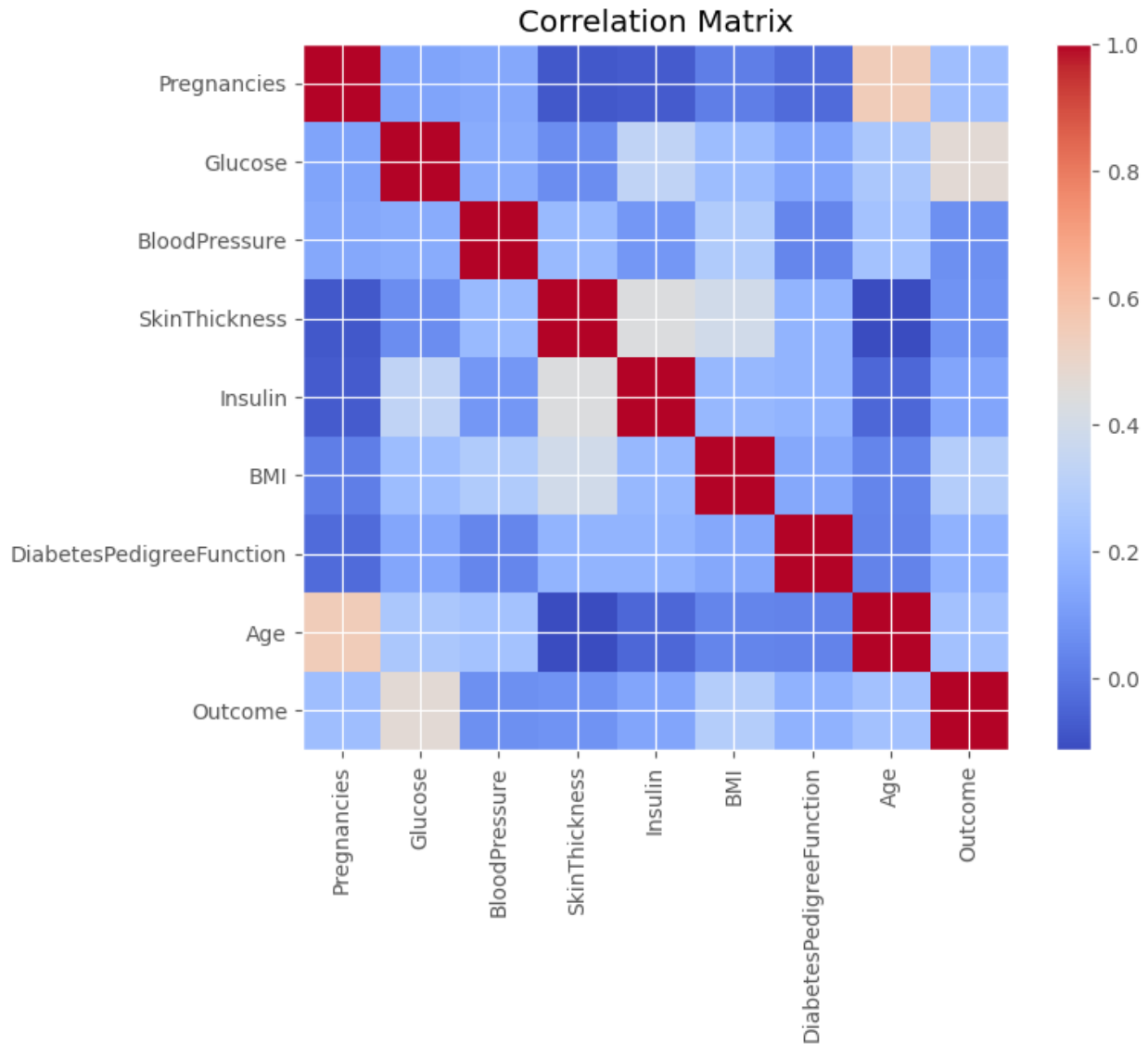
memory usage: 54.1 KB

Feature Correlation Heatmap & Target Association

```
corr = df.corr()

plt.figure(figsize=(8, 6))
plt.imshow(corr, cmap='coolwarm', interpolation='nearest')
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotation=90)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title("Correlation Matrix")
plt.show()

print("Correlation with Outcome:")
print(corr["Outcome"].sort_values(ascending=False))
```



```
Correlation with Outcome:
Outcome      1.000000
Glucose      0.466581
BMI          0.292695
Age          0.238356
Pregnancies  0.221898
DiabetesPedigreeFunction 0.173844
Insulin      0.130548
SkinThickness 0.074752
BloodPressure 0.065068
Name: Outcome, dtype: float64
```

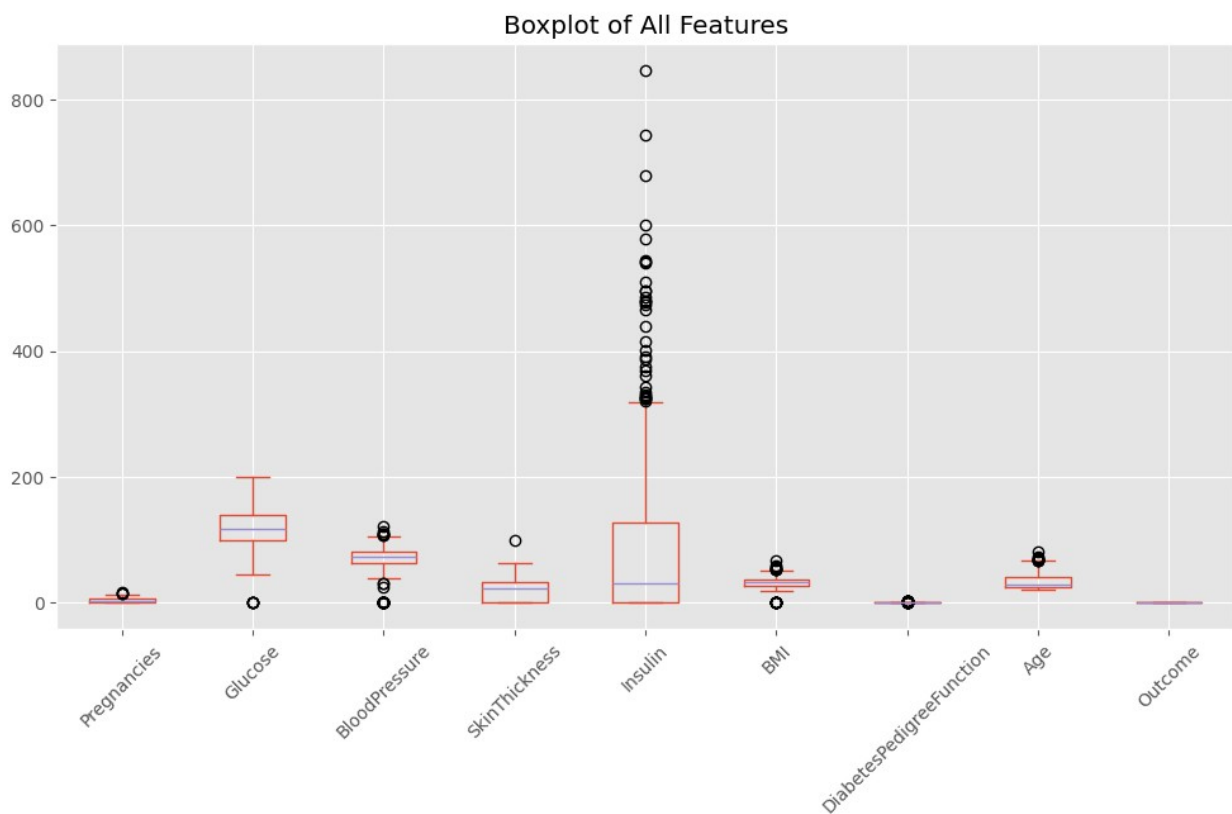
Feature Distributions & Class Balance Visualizations

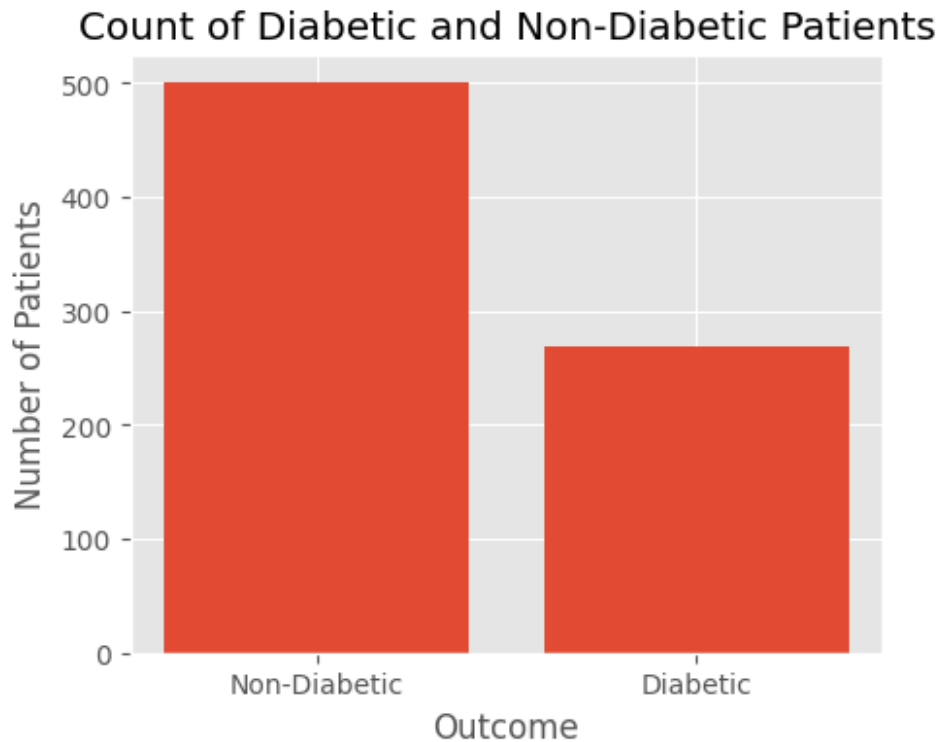
```

# Feature Boxplots
df.plot(kind='box', figsize=(12, 6))
plt.title("Boxplot of All Features")
plt.xticks(rotation=45)
plt.grid(True)
plt.show()

# Target Class Distribution
counts = df["Outcome"].value_counts()
plt.figure(figsize=(5, 4))
plt.bar(["Non-Diabetic", "Diabetic"], counts)
plt.title("Count of Diabetic and Non-Diabetic Patients")
plt.xlabel("Outcome")
plt.ylabel("Number of Patients")
plt.show()

```





Zero-Value Inspection & Invalid Zero Replacement

```
columns_with_zero = ['Glucose', 'BloodPressure', 'SkinThickness',  
                     'Insulin', 'BMI']  
  
print("Number of 0 values before replacement:\n")  
for col in columns_with_zero:  
    print(f"{col}: {(df[col] == 0).sum()}")  
  
# Replace 0s with NaN for biologically invalid zero measurements  
df[columns_with_zero] = df[columns_with_zero].replace(0, np.nan)  
  
print("\nNumber of NaN values after replacing 0s:")  
print(df.isnull().sum())
```

Number of 0 values before replacement:

Glucose: 5
BloodPressure: 35
SkinThickness: 227
Insulin: 374
BMI: 11

Number of NaN values after replacing 0s:

Pregnancies	0
Glucose	5
BloodPressure	35

```
SkinThickness      227
Insulin            374
BMI                11
DiabetesPedigreeFunction  0
Age                0
Outcome            0
dtype: int64
```

Median Imputation

```
for col in columns_with_zero:
    df[col].fillna(df[col].median(), inplace=True)

print("\nNumber of NaN values after median imputation:")
print(df.isnull().sum())
```

```
Number of NaN values after median imputation:
Pregnancies      0
Glucose          0
BloodPressure    0
SkinThickness    0
Insulin          0
BMI              0
DiabetesPedigreeFunction  0
Age              0
Outcome          0
dtype: int64
```

Feature Scaling & Dataset Splitting

```
X = df.drop('Outcome', axis=1)
y = df['Outcome']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)

# Split into train and test sets with stratification
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled_df, y, test_size=0.2, random_state=42, stratify=y
)

print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

print("\nClass distribution in training set:")
```

```
print(pd.Series(y_train).value_counts())
print("\nClass distribution in test set:")
print(pd.Series(y_test).value_counts(normalize=True))
```

```
X_train shape: (614, 8)
X_test shape: (154, 8)
y_train shape: (614,)
y_test shape: (154,)
```

Class distribution in training set:

Outcome

0 400

1 214

Name: count, dtype: int64

Class distribution in test set:

Outcome

0 0.649351

1 0.350649

Name: proportion, dtype: float64

Verify & Handle Residual Missing Values

```
# Impute any residual NaNs within train/test sets if present
X_train = X_train.fillna(X_train.median(numeric_only=True))
X_test = X_test.fillna(X_test.median(numeric_only=True))

print("Missing values in X_train:", X_train.isnull().sum().sum())
print("Missing values in X_test:", X_test.isnull().sum().sum())
```

Missing values in X_train: 0

Missing values in X_test: 0

Baseline Logistic Regression Model

```
model_base = LogisticRegression(random_state=42)
model_base.fit(X_train, y_train)

print("Baseline Logistic Regression Model trained successfully")
```

Baseline Logistic Regression Model trained successfully

Baseline Model Evaluation

```
y_pred_base = model_base.predict(X_test)
y_pred_proba_base = model_base.predict_proba(X_test)[:, 1]

conf_matrix_base = confusion_matrix(y_test, y_pred_base)
print("Confusion Matrix (Baseline):")
```

```

print(conf_matrix_base)

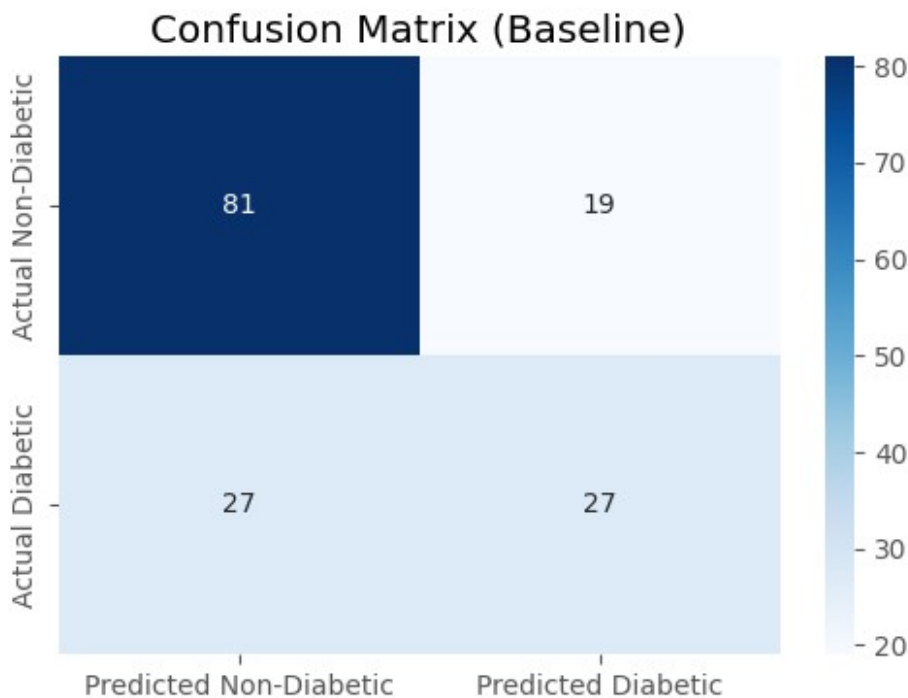
plt.figure(figsize=(6, 4))
sns.heatmap(
    conf_matrix_base, annot=True, fmt="d", cmap="Blues",
    xticklabels=['Predicted Non-Diabetic', 'Predicted Diabetic'],
    yticklabels=['Actual Non-Diabetic', 'Actual Diabetic']
)
plt.title("Confusion Matrix (Baseline)")
plt.show()

acc_base = accuracy_score(y_test, y_pred_base)
prec_base = precision_score(y_test, y_pred_base)
rec_base = recall_score(y_test, y_pred_base)
f1_base = f1_score(y_test, y_pred_base)
roc_base = roc_auc_score(y_test, y_pred_proba_base)

print(f"Accuracy: {acc_base:.4f}")
print(f"Precision: {prec_base:.4f}")
print(f"Recall: {rec_base:.4f}")
print(f"F1 Score: {f1_base:.4f}")
print(f"ROC AUC Score: {roc_base:.4f}")

Confusion Matrix (Baseline):
[[81 19]
 [27 27]]

```



Accuracy: 0.7013
Precision: 0.5870
Recall: 0.5000
F1 Score: 0.5400
ROC AUC Score: 0.8128

Apply SMOTE & Train Resampled Model

```
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train,
y_train)

print("Class distribution after SMOTE:")
print(pd.Series(y_train_resampled).value_counts())

model_resampled = LogisticRegression(random_state=42)
model_resampled.fit(X_train_resampled, y_train_resampled)

print("\nLogistic Regression Model (Resampled) trained successfully")
```

Class distribution after SMOTE:

Outcome	
0	400
1	400

Name: count, dtype: int64

Logistic Regression Model (Resampled) trained successfully

Resampled Model Evaluation

```
y_pred_smote = model_resampled.predict(X_test)
y_pred_proba_smote = model_resampled.predict_proba(X_test)[: , 1]

conf_matrix_smote = confusion_matrix(y_test, y_pred_smote)
print("Confusion Matrix (SMOTE Model):")
print(conf_matrix_smote)

plt.figure(figsize=(6, 4))
sns.heatmap(
    conf_matrix_smote, annot=True, fmt="d", cmap="Greens",
    xticklabels=['Predicted Non-Diabetic', 'Predicted Diabetic'],
    yticklabels=['Actual Non-Diabetic', 'Actual Diabetic']
)
plt.title("Confusion Matrix (SMOTE Resampled Model)")
plt.show()

acc_smote = accuracy_score(y_test, y_pred_smote)
prec_smote = precision_score(y_test, y_pred_smote)
rec_smote = recall_score(y_test, y_pred_smote)
```

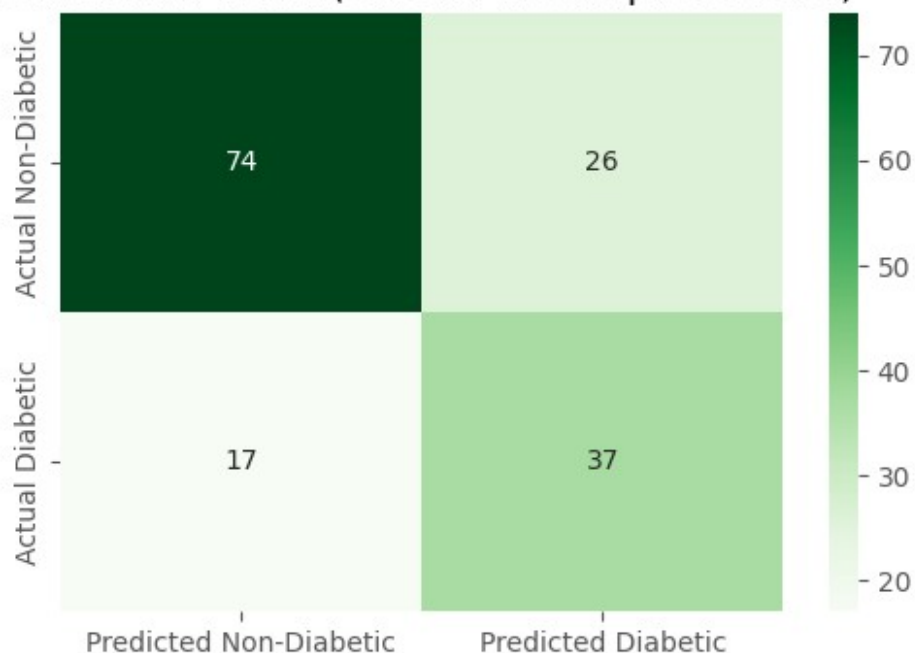
```
f1_smote = f1_score(y_test, y_pred_smote)
roc_smote = roc_auc_score(y_test, y_pred_proba_smote)

print(f"Accuracy: {acc_smote:.4f}")
print(f"Precision: {prec_smote:.4f}")
print(f"Recall: {rec_smote:.4f}")
print(f"F1 Score: {f1_smote:.4f}")
print(f"ROC AUC Score: {roc_smote:.4f}")
```

Confusion Matrix (SMOTE Model):

```
[[74 26]
 [17 37]]
```

Confusion Matrix (SMOTE Resampled Model)



```
Accuracy: 0.7208
Precision: 0.5873
Recall: 0.6852
F1 Score: 0.6325
ROC AUC Score: 0.8109
```

Feature Importance (Logistic Regression Coefficients)

```
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model_resampled.coef_[0]
}).sort_values(by='Coefficient', ascending=False)
```

```

print("\nFeature Importance (Resampled Model):\n")
print(feature_importance)

plt.figure(figsize=(10, 6))
sns.barplot(x='Coefficient', y='Feature', data=feature_importance,
palette='viridis')
plt.title('Logistic Regression Feature Coefficients (Resampled Model)')
plt.xlabel('Coefficient Value')
plt.ylabel('Feature')
plt.show()

```

Feature Importance (Resampled Model):

	Feature	Coefficient
1	Glucose	1.272081
5	BMI	0.766550
0	Pregnancies	0.293924
6	DiabetesPedigreeFunction	0.251101
7	Age	0.148011
3	SkinThickness	0.062731
2	BloodPressure	-0.056352
4	Insulin	-0.120413



Performance Metrics Comparison (Before vs. After SMOTE)

```

comparison_data = {
    'Metric': ['Accuracy', 'Precision', 'Recall', 'F1 Score', 'ROC AUC']
}

```

```
Score'],
    'Before SMOTE': [acc_base, prec_base, rec_base, f1_base,
roc_base],
    'After SMOTE': [acc_smote, prec_smote, rec_smote, f1_smote,
roc_smote]
}

comparison_df = pd.DataFrame(comparison_data)

print("Comparison of Model Metrics (Before vs. After SMOTE):")
print(comparison_df.round(4))
```

```
Comparison of Model Metrics (Before vs. After SMOTE):
```

	Metric	Before SMOTE	After SMOTE
0	Accuracy	0.7013	0.7208
1	Precision	0.5870	0.5873
2	Recall	0.5000	0.6852
3	F1 Score	0.5400	0.6325
4	ROC AUC Score	0.8128	0.8109

High-Risk Non-Diabetic Subgroup Analysis

```
# Combine actual outcomes, predicted probabilities, and original
features
results_df = pd.DataFrame({
    'Actual_Outcome': y_test,
    'Predicted_Probability': y_pred_proba_smote
}).reset_index(drop=True)

original_X_test_df = pd.DataFrame(X_test,
columns=X.columns).reset_index(drop=True)
results_df = pd.concat([results_df, original_X_test_df], axis=1)

# Cases where actual outcome is non-diabetic (0) but predicted
probability > 0.5
high_risk_non_diabetic = results_df[
    (results_df['Actual_Outcome'] == 0) &
    (results_df['Predicted_Probability'] > 0.5)
]

print("\nPatient records with high-risk probabilities despite being
classified as non-diabetic:")
print(high_risk_non_diabetic.head())

print("\nSummary statistics for high-risk non-diabetic patients:")
print(high_risk_non_diabetic.describe())
```

```
Patient records with high-risk probabilities despite being classified
as non-diabetic:
```

	Actual_Outcome	Predicted_Probability	Pregnancies	Glucose \
0	0	0.737877	0.936914	1.227667
9	0	0.932256	-1.141852	1.424916
16	0	0.810598	1.827813	1.326292
18	0	0.901781	-0.250952	2.279660
24	0	0.688832	0.936914	0.109925

	BloodPressure	SkinThickness	Insulin	BMI \
0	-0.693761	-0.012301	-0.181541	-0.735763
9	0.298896	1.581234	1.324364	2.247921
16	0.960667	-0.012301	-0.181541	-0.692100
18	-0.362876	-1.605837	-0.123622	-0.226354
24	1.126110	-0.012301	-0.181541	0.748802

	DiabetesPedigreeFunction	Age
0	-0.537208	0.575118
9	-0.642912	-0.616111
16	-0.875461	1.766346
18	-0.522107	0.064591
24	-0.507006	1.511083

Summary statistics for high-risk non-diabetic patients:

	Actual_Outcome	Predicted_Probability	Pregnancies	Glucose \
count	26.0	26.000000	26.000000	26.000000
mean	0.0	0.742853	0.308715	0.782593
std	0.0	0.137609	1.161824	0.855868
min	0.0	0.506046	-1.141852	-0.613320
25%	0.0	0.658336	-0.770643	0.151018
50%	0.0	0.744164	0.046014	0.701671
75%	0.0	0.836890	0.936914	1.301635
max	0.0	0.947908	2.718712	2.378284

	BloodPressure	SkinThickness	Insulin	BMI \
count	26.000000	26.000000	26.000000	26.000000
mean	0.435704	0.442995	0.669875	0.677149
std	0.913970	1.090336	1.694375	1.047572
min	-1.851862	-2.061133	-0.413219	-0.954082
25%	-0.156072	-0.012301	-0.181541	-0.153581
50%	0.381617	0.329171	-0.181541	0.734247
75%	1.105430	1.325130	0.504804	1.196354
max	1.870603	2.833298	6.247517	2.888322

	DiabetesPedigreeFunction	Age
count	26.000000	26.000000
mean	-0.245418	0.146407
std	0.603682	1.084455
min	-1.044587	-1.041549
25%	-0.682173	-0.616111
50%	-0.469255	-0.360847
75%	0.227638	0.724021
max	1.305065	2.872487

Prepared by 24CS099



Faculty of Technology and Engineering

Chandubhai S. Patel Institute of Technology

Department of Computer Science & Engineering

Date: 08/08/2026

Practical Performa

Academic Year	:	2026-27	Semester	:	5 th
Course code	:	CSUC301	Course name	:	Machine Learning

Practical- No. 3

Aim: Implement a linear regression:

Implement a logistic regression: A healthcare organization aims to identify individuals who are at high risk of developing diabetes using patient health measurements collected during routine medical examinations. The organization requires a predictive system that can estimate the likelihood of diabetes occurrence and support early intervention strategies. Develop an solution capable of assessing risk, handling challenges in the dataset, and providing reliable decision support for healthcare professionals..

- Questions and Answers :

1.	Which patient attributes appear to have the strongest influence on diabetes occurrence? →Based on the logistic regression coefficients, Glucose clearly emerges as the single strongest predictor of diabetes occurrence. It is followed closely by BMI, Pregnancies, and Diabetes Pedigree Function, all of which display substantial positive influence on the target outcome. Conversely, attributes like Skin Thickness and Blood Pressure show minimal to slightly negative predictive weights. Overall, metabolic indicators related to blood sugar levels and body mass index dominate the model's decision-making process.
----	---

2.	Is the dataset balanced across outcome classes? What impact could class imbalance have on prediction quality? → No, the dataset is noticeably imbalanced, with roughly 65% of individuals classified as non-diabetic and only 35% as diabetic. Left unaddressed, this skew causes a machine learning model to favor the majority class, predicting "non-diabetic" more often to artificially inflate overall accuracy. → Consequently, the model suffers from poor sensitivity, missing a high proportion of actual diabetic patients. Oversampling techniques like SMOTE help balance training data to ensure the minority class receives equal emphasis during learning.
3.	How effectively can the developed solution distinguish diabetic and non-diabetic patients? Justify using appropriate evaluation measures [confusion matrix, accuracy, precision, recall, F1-score] → After applying SMOTE, the developed logistic regression model demonstrates a moderate ability to separate diabetic from non-diabetic cases. Across the test set, it achieves an overall accuracy of about 71–72%. Looking at the confusion matrix, it correctly identifies 74 true negatives and 37 true positives, while producing 26 false positives and 17 false negatives. With a precision of ~0.58, a recall of ~0.67, and an F1-score around 0.62, the solution shows strong minority-class recovery but still leaves room for reducing false alarms.
4.	Which evaluation measures provide the most reliable assessment for this healthcare problem? → In healthcare risk assessment, Recall (Sensitivity) and the F1-Score provide far more reliable metrics than simple accuracy. Accuracy can easily mask poor performance when dealing with imbalanced medical datasets. Recall is particularly critical because it directly captures the proportion of actual positive cases successfully detected by the system. Pairing it with F1-Score ensures a balanced evaluation, holding the model accountable for both missing sick patients and generating excessive false alarms.
5.	Which patient records receive high-risk probabilities despite being classified as non diabetic, and what insights can be derived from such cases? → Patients receiving high predicted probabilities despite being labeled non-diabetic (Actual Outcome = 0, Probability > 0.5) represent borderline or early-stage risk profiles. Although not yet officially diagnosed as diabetic, their summary statistics reveal elevated average values for key features like Glucose and BMI compared to typical healthy individuals. These cases suggest that while clinical labels are binary, diabetes risk operates on a continuous spectrum. Identifying these high-risk non-diabetic individuals creates a valuable window for early lifestyle interventions before full disease onset.

6.	What are the consequences of false positives and false negatives in a healthcare setting? → In a medical setting, false negatives carry severe clinical consequences because delayed diagnoses allow disease progression, increasing long-term complication risks and treatment costs. On the flip side, false positives trigger unnecessary patient anxiety, additional diagnostic testing, and potential exposure to unneeded treatments. While both misclassifications impose burdens on healthcare infrastructure, false negatives are generally considered far more dangerous due to direct threats to patient health. Together, they highlight why medical AI must carefully balance precision and sensitivity.
7.	Which visualizations best explain the predictive performance of the developed solution? → A Confusion Matrix heatmap is the most effective visualization because it plainly lays out true positives, true negatives, false positives, and false negatives in a single grid. Complementing this with a Feature Importance / Coefficient Bar Chart clearly demonstrates which biological factors drive those predictions. Furthermore, an ROC Curve illustrates how well the model discriminates across various classification thresholds. Together, these visualizations make model behavior transparent and intuitive for both technical reviewers and medical professionals.